Aircraft Positioning in a Hangar

Technical Documentation of the Optimisation Model and Solution Methods

Baobab Soluciones

May 14, 2026

Contents

1 Problem Statement

1.1 Context

An aircraft maintenance hangar contains a fixed set of *positions* $\mathcal{P} = \{p_1, \ldots, p_P\}$ where aircraft can be parked and serviced. Each position can hold at most one aircraft at a time. A set of aircraft $\mathcal{R} = \{r_1, \ldots, r_R\}$ must be scheduled, each requiring a sequence of maintenance jobs carried out at a single position for its entire stay.

The hangar has a physical access topology: some positions can only be reached by driving through other positions. This creates *blocking arcs*: if aircraft r occupies a *front* position p while aircraft r' needs to enter or exit a *rear* position p', a ground movement is needed, generating delay and logistical cost.

1.2 Input Data

Aircraft. Each aircraft $r \in \mathcal{R}$ is characterised by:

- $e_r \geq 0$: earliest time at which r can enter the hangar.
- $T_r > 0$: target finish time (when the customer expects the aircraft back in service). Finishing after T_r incurs a delay penalty.
- A client identifier (for multi-client instances).

Jobs. Each aircraft r has an ordered sequence of maintenance jobs $\mathcal{J}^r = (j_1^r, j_2^r, \ldots)$ that must be executed *sequentially* at the assigned position:

- $d_j > 0$: processing duration of job j.
- Precedences: j must finish before j' starts whenever $(j, j') \in \mathcal{E}$. Within a single aircraft the jobs form a chain; cross-aircraft precedences are also possible.

The *total duration* of aircraft r is $D_r = \sum_{j \in \mathcal{J}^r} d_j$.

Hangar topology. The blocking structure is a directed acyclic graph (DAG):

$$\mathcal{B} \subseteq \mathcal{P} \times \mathcal{P}, \quad (p, p') \in \mathcal{B} \;\Rightarrow\; \text{an aircraft in } p \text{ blocks access to } p'.$$

A position's *blocking load* is $\ell(p) = |\{p' : p \text{ can reach } p' \text{ transitively in } \mathcal{B}\}|$.

Separation. A minimum time gap $\delta \geq 0$ must be observed between consecutive aircraft occupying the same position: if aircraft r finishes at f_r in position p, the next aircraft r' assigned to p must start no earlier than $f_r + \delta$.

1.3 Decisions

1. **Assignment**: which position $p \in \mathcal{P}$ is assigned to aircraft r. Each aircraft occupies exactly one position.
2. **Scheduling**: start time s_j for each job j, respecting precedences, earliest-start, and non-overlap within each position.

1.4 Objective Function

The objective is a weighted sum of three KPIs:

$$\min \quad w_M \cdot M + w_D \cdot \sum_{r \in \mathcal{R}} \Delta_r + w_V \cdot V \tag{1}$$

where:

Symbol	Definition	Default weight
M	Makespan: $\max_r f_r$ (time until the last aircraft leaves)	$w_M = 0.1$
Δ_r	Individual delay: $\max(0,\ f_r - T_r)$	$w_D = 1.0$
V	Total blocking movements count	$w_V = 10$

The weights above correspond to the experimental configuration used in this project. The MILP model uses higher values ($w_M = 10$, $w_D = 100$, $w_V = 1$) to avoid numerical issues; all heuristics use the same weights as the MILP when called from the experiment runner.

1.5 Blocking Movements

A *blocking event* occurs when aircraft r arrives at or departs from rear position p' while aircraft r_b is occupying a front position p such that $(p, p') \in \mathcal{B}$ and their time windows overlap.

Formally, let s_r, f_r denote the start and finish of aircraft r:

- **Entry block**: r' enters p' while $r_b \in p$: $s_{r'} \in (s_{r_b},\ f_{r_b} - \delta)$.
- **Exit block**: r' exits p' while $r_b \in p$: $f_{r'} \in (s_{r_b},\ f_{r_b} - \delta)$.

Each detected blocking event contributes 2 to V (one movement in, one out).

1.6 Feasibility

A solution is *feasible* if:

1. Every aircraft is assigned to exactly one position.
2. Jobs of an aircraft are scheduled at the assigned position, in order, without gaps within the position chain.
3. No two aircraft overlap at the same position (minimum separation δ between consecutive aircraft).
4. Each aircraft's first job starts no earlier than e_r.
5. All job precedences $(j, j') \in \mathcal{E}$ are satisfied.

2 MILP Formulation

The exact model is implemented in `models/milp_pyomo.py` as a Pyomo abstract model solved with Gurobi.

2.1 Sets

Set	Description
$\mathcal{J}$	All jobs
$\mathcal{R}$	All aircraft
$\mathcal{P}$	All positions
$\mathcal{C}$	Clients
$\mathcal{B} \subseteq \mathcal{P} \times \mathcal{P}$	Blocking arcs (p, p')
$\mathcal{E} \subseteq \mathcal{J} \times \mathcal{J}$	Job precedences (j, j')
$\mathcal{T} = \mathcal{R} \times \mathcal{R} \times \mathcal{B}$	Blocking tuples (r, r', p, p'), $r \neq r'$

2.2 Parameters

Parameter	Domain	Description
d_j	$\mathbb{R}_{\geq 0}$	Duration of job j
e_r	$\mathbb{R}_{\geq 0}$	Earliest start for aircraft r
T_r	$\mathbb{R}_{\geq 0}$	Target finish time for aircraft r
R_{jr}	$\{0, 1\}$	1 if job j belongs to aircraft r
F_{jr}	$\{0, 1\}$	1 if j is the first job of r
L_{jr}	$\{0, 1\}$	1 if j is the last job of r
δ	$\mathbb{R}_{\geq 0}$	Minimum separation between consecutive aircraft
M	$\mathbb{R}_{\geq 0}$	Big-M constant ($\max_r T_r + \sum_j d_j$)
w_M, w_D, w_V	$\mathbb{R}_{\geq 0}$	Objective weights

2.3 Decision Variables

Variable	Domain	Description
s_j	$\mathbb{R}_{\geq 0}$	Start time of job j
f_j	$\mathbb{R}_{\geq 0}$	Finish time of job j
S_r	$\mathbb{R}_{\geq 0}$	Start time of aircraft r (first job)
F_r	$\mathbb{R}_{\geq 0}$	Finish time of aircraft r (last job)
x_{rp}	$\{0, 1\}$	1 if aircraft r is assigned to position p
y_{jp}	$\{0, 1\}$	1 if job j is at position p
$z_{jj'p}$	$\{0, 1\}$	1 if job j precedes j' at position p
Δ_r	$\mathbb{R}_{\geq 0}$	Delay of aircraft r
$\hat{M}$	$\mathbb{R}_{\geq 0}$	Makespan
$\hat{V}$	$\mathbb{R}_{\geq 0}$	Total movements
Auxiliary blocking variables (per tuple $(r, r', p, p') \in \mathcal{T}$):		
$\alpha^{\text{in}}_{rr'pp'}$	$\{0, 1\}$	r' enters p' after r enters p
$\beta^{\text{in}}_{rr'pp'}$	$\{0, 1\}$	r' enters p' before r leaves p (minus δ)
$u^{\text{in}}_{rr'pp'}$	$\{0, 1\}$	Entry blocking indicator ($= x_{rp} \cdot x_{r'p'} \cdot \alpha^{\text{in}} \cdot \beta^{\text{in}}$)
$\alpha^{\text{out}}_{rr'pp'}$	$\{0, 1\}$	r' exits p' after r enters p
$\beta^{\text{out}}_{rr'pp'}$	$\{0, 1\}$	r' exits p' before r leaves p (minus δ)
$u^{\text{out}}_{rr'pp'}$	$\{0, 1\}$	Exit blocking indicator

2.4 Objective

$$\min \quad w_M \cdot \hat{M} \;+\; w_D \cdot \sum_{r \in \mathcal{R}} \Delta_r \;+\; w_V \cdot \hat{V} \tag{2}$$

2.5 Constraints

Assignment and timing.

$$\sum_{p \in \mathcal{P}} x_{rp} = 1 \qquad \forall r \in \mathcal{R} \quad \text{(each aircraft assigned to exactly one position)} \tag{3}$$

$$y_{jp} = \sum_{r \in \mathcal{R}} R_{jr} \cdot x_{rp} \qquad \forall j \in \mathcal{J},\ p \in \mathcal{P} \quad \text{(job inherits aircraft position)} \tag{4}$$

$$f_j = s_j + d_j \qquad \forall j \in \mathcal{J} \tag{5}$$

$$s_j \geq e_r \cdot F_{jr} \qquad \forall (j,r) : F_{jr} = 1 \quad \text{(earliest start)} \tag{6}$$

$$S_r = \sum_{j \in \mathcal{J}} F_{jr} \cdot s_j \qquad \forall r \in \mathcal{R} \tag{7}$$

$$F_r = \sum_{j \in \mathcal{J}} L_{jr} \cdot f_j \qquad \forall r \in \mathcal{R} \tag{8}$$

Precedences.

$$s_{j'} \geq f_j \qquad \forall (j,j') \in \mathcal{E} \tag{9}$$

Non-overlap within a position (disjunctive scheduling). Binary variable $z_{jj'p} = 1$ means job j is scheduled before job j' at position p. Using Big-M linearisation:

$$s_{j'} \geq f_j - M\big(1 - z_{jj'p} + (1 - y_{jp}) + (1 - y_{j'p})\big) \qquad \forall j \neq j',\ p \tag{10}$$

$$s_j \geq f_{j'} - M\big(z_{jj'p} + (1 - y_{jp}) + (1 - y_{j'p})\big) \qquad \forall j \neq j',\ p \tag{11}$$

$$z_{jj'p} + z_{j'jp} \geq y_{jp} + y_{j'p} - 1 \qquad \forall j < j',\ p \tag{12}$$

$$z_{jj'p} + z_{j'jp} \leq 1 \qquad \forall j < j',\ p \tag{13}$$

Aircraft-level non-overlap — one of the two orderings must hold:

$$\sum_{\substack{j,j' \in \mathcal{J} \\ j \neq j'}} \big(L_{jr} \cdot F_{j'r'} \cdot z_{jj'p} + L_{j'r'} \cdot F_{jr} \cdot z_{j'jp}\big) \geq x_{rp} + x_{r'p} - 1 \qquad \forall r < r',\ p \tag{14}$$

Minimum separation δ between consecutive aircraft at the same position. Let $q_{rr'p} = \sum_{j,j' :\, j \neq j'} L_{jr}\, F_{j'r'}\, z_{jj'p}$ be the indicator that r precedes r' at position p. Then:

$$S_{r'} \geq F_r + \delta - M(1 - q_{rr'p}) - M(1 - x_{rp}) - M(1 - x_{r'p}) \qquad \forall r < r',\ p \tag{15}$$

$$S_r \geq F_{r'} + \delta - M(1 - q_{r'rp}) - M(1 - x_{rp}) - M(1 - x_{r'p}) \qquad \forall r < r',\ p \tag{16}$$

These two constraints were absent from the original formulation (2026-05-14 audit): the original model packed consecutive aircraft with zero gap, while the heuristic enforced $\delta = 10$. After correction, both methods use the same feasibility definition.

Blocking movement indicators. For each tuple $(r, r', p, p') \in \mathcal{T}$ with $(p, p') \in \mathcal{B}$:

Entry block (r' enters p' while r is in p):

$$S_{r'} \geq S_r - M(1 - \alpha^{\text{in}}) \tag{17}$$
$$S_{r'} \leq S_r + M \cdot \alpha^{\text{in}} \tag{18}$$
$$S_{r'} \leq F_r - \delta + M(1 - \beta^{\text{in}}) \tag{19}$$
$$S_{r'} \geq F_r - M \cdot \beta^{\text{in}} - M(1 - x_{rp}) - M(1 - x_{r'p'}) \tag{20}$$

The entry blocking indicator is the linearisation of $u^{\text{in}} = x_{rp} \cdot x_{r'p'} \cdot \alpha^{\text{in}} \cdot \beta^{\text{in}}$:

$$u^{\text{in}} \leq x_{rp}, \quad u^{\text{in}} \leq x_{r'p'}, \quad u^{\text{in}} \leq \alpha^{\text{in}}, \quad u^{\text{in}} \leq \beta^{\text{in}} \tag{21}$$
$$u^{\text{in}} \geq x_{rp} + x_{r'p'} + \alpha^{\text{in}} + \beta^{\text{in}} - 3 \tag{22}$$

Exit block (r' exits p' while r is in p): Analogous constraints with $F_{r'}$ replacing $S_{r'}$, yielding u^{out}.

KPI aggregation.

$$\hat{V} = 2 \sum_{(r,r',p,p') \in \mathcal{T}} \left(u^{\text{in}}_{rr'pp'} + u^{\text{out}}_{rr'pp'} \right) \tag{23}$$
$$\Delta_r \geq \sum_{j \in \mathcal{J}} L_{jr}(f_j - T_r) \qquad \forall r \in \mathcal{R} \tag{24}$$
$$\hat{M} \geq f_j \qquad \forall j \in \mathcal{J} \tag{25}$$

2.6 Model Size

For an instance with R aircraft, J jobs, P positions, and B blocking arcs the main variable and constraint counts are:

Component	Order of magnitude
Continuous variables	$O(J + R)$
Binary variables (x, y, z)	$O(JP + J^2P)$
Blocking binary variables (α, β, u)	$O(R^2B)$
Non-overlap constraints	$O(J^2P)$
Blocking constraints	$O(R^2B)$

2.7 Solver Configuration

The model is solved with **Gurobi** via Pyomo's `SolverFactory`. Relevant options:

Option	Type	Effect
`TimeLimit`	seconds	Wall-clock budget
`MIPGap`	fraction	Stop when gap $\leq$ this value (0 = optimal)
`NoRelHeurTime`	seconds	Aggressive primal heuristics before LP relaxation

3 Constructive Heuristic

3.1 Overview

The constructive heuristic (`solvers/constructive_heuristic.py`) builds solutions using a **Greedy Randomised Adaptive Search Procedure (GRASP)** combined with an optional **Iterated Local Search (ILS)** perturbation mechanism. Each iteration constructs a complete feasible solution and polishes it with a local search; the best solution found across all iterations is retained.

By default (`ils_ratio = 0.0`) the heuristic runs as pure GRASP — each iteration starts from an empty schedule. Setting `ils_ratio > 0` enables ILS: a fraction of iterations perturb the current best solution instead of building from scratch.

3.2 Algorithm

Algorithm 1 Constructive Heuristic (GRASP + ILS)

$s^* \leftarrow \emptyset$, $z^* \leftarrow \infty$
while time budget not exhausted **do**
  if $s^* \neq \emptyset$ **and** Uniform$(0,1) < \rho_{\text{ils}}$ **then**
    Remove n_{perturb} aircraft from s^*; keep the rest fixed
  else
    Start from an empty schedule
  end if
  Schedule fixed aircraft first (in earliest-start order)
  while unscheduled aircraft remain **do**
    Evaluate all (r, p) pairs over unscheduled aircraft and all positions
    Score each pair; sort by score; assign GRASP weights $w_i = (1-\alpha)^i$
    Sample one pair (r^*, p^*); commit assignment; remove r^* from pool
  end while
  if iteration $\equiv 0 \pmod{ls\_every}$ **or** current obj $< z^*$ **then**
    $s \leftarrow \text{LocalSearch}(s)$
  end if
  if $z(s) < z^*$ **then** $s^* \leftarrow s$; $z^* \leftarrow z(s)$
  end if
end while
return s^*

Joint pair selection. At each insertion step the heuristic enumerates *all* remaining (aircraft, position) pairs, not just the positions for a fixed aircraft. This means aircraft and positions are chosen simultaneously: one draw selects both who goes next and where they go. The sorted list may contain $|\mathcal{R}_{\text{free}}| \times |\mathcal{P}|$ entries; the geometric weights favour the globally cheapest pair regardless of aircraft identity.

3.3 Construction Scoring

For each candidate pair (r, p), the start time $t_s(r, p)$ is obtained from `_find_best_start`, which compares two candidate times and returns the one with lower estimated cost:

- t_{imm}: immediate start — position free time plus minimum separation δ. May create new blocking events.
- t_{nob}: no-block start — iteratively advances t_s past every blocking conflict until none remain.

The cost comparison is:

$$c(t_s,\ \text{extra\_movs}) = w_D \cdot \max(0, t_s + D_r - T_r) + w_M \cdot (t_s + D_r) + w_V \cdot \text{extra\_movs} \cdot 2$$

If $c(t_{\text{imm}}, \text{movs}_{\text{imm}}) \leq c(t_{\text{nob}}, 0)$, the immediate start is used despite the extra movements.

The GRASP score for the pair then adds a gap penalty:

$$\text{score}(r, p) = w_D \cdot \Delta_r + w_M \cdot t_f + w_V \cdot V' \cdot 2 + w_{\text{gap}} \cdot g \tag{26}$$

where V' is the total movement count after tentatively inserting r at p, and $g = \max(0,\ t_s - \texttt{pos\_free\_at}[p])$ is the idle gap created at position p (penalised only when p has already been used at least once).

The GRASP rank-based probability follows a geometric distribution:

$$P(\text{select rank } i) \ \propto\ (1-\alpha)^i, \quad i = 0, 1, 2, \ldots \tag{27}$$

$\alpha = 0$ gives **uniform random** selection (all weights equal to 1); $\alpha = 1$ gives **fully greedy** selection (only rank 0 has non-zero weight). The default value is $\alpha = 0.3$, which strongly favours top candidates while retaining stochastic diversity.

3.4 Local Search

After each construction (run every `ls_every` iterations, and always when a new best is found) the solution is polished with three operators applied in priority order until no improvement is found:

1. **Single-move**: try reassigning each aircraft to every other position; accept the first improvement.
2. **2-opt swap**: try swapping positions of every pair of aircraft with different positions; accept the first improvement.
3. **Intra-position adjacent swap**: for each position with ≥ 2 aircraft, try swapping the processing order of each pair of consecutive aircraft; accept the first improvement.

Operators are tried in the order listed; any improvement restarts from operator 1. The search is fully convergent (runs until no operator finds an improvement). Each candidate is evaluated by rebuilding the schedule in $O(R)$, giving $O(R^2 P)$ cost per pass.

3.5 ILS Perturbation

When `ils_ratio` > 0, with probability ρ_{ils} per iteration the heuristic *perturbs* the current best solution instead of building from scratch:

1. Sample $k = \min(n_{\text{perturb}},\ R - 1)$ aircraft to remove.
2. Fix the remaining $R - k$ aircraft at their current positions; schedule them in earliest-start order.
3. Re-insert the k removed aircraft using the standard GRASP loop.

3.6 Parameters

Parameter	Default	Description
`time_limit_s`	60	Wall-clock budget (seconds)
`alpha`	0.3	GRASP geometric decay
`min_separation`	10.0	Minimum gap between consecutive aircraft
`weight_makespan`	0.1	w_M
`weight_delay`	1.0	w_D
`weight_movements`	10	w_V
`weight_gap`	$= w_M$	Penalty for position idle time
`n_perturb`	4	Aircraft removed per ILS kick
`ils_ratio`	0.0	Fraction of iterations using ILS warm-start (0 = pure GRASP)
`ls_every`	1	Run local search every N iterations
`seed`	None	Random seed

3.7 Complexity

Each GRASP step evaluates $|\mathcal{R}_{\text{free}}| \times P$ pairs. Summed over all R insertion steps: $O(R^2P)$ evaluations, each requiring $O(R)$ for movement counting, giving $O(R^3P)$ per construction. Local search (first improvement, fully convergent) adds $O(R^2P)$ per pass.

4 Large Neighbourhood Search (LNS)

4.1 Overview

The LNS solver (`solvers/lns_solver.py`) iterates a **destroy–repair–accept** cycle. Starting from a good initial solution, it repeatedly removes a subset of aircraft (*destroy*), re-optimises their position assignments (*repair*), and accepts the result if it improves the incumbent.

Three repair operators are available:

- *Exhaustive repair* — enumerate all $|\mathcal{P}|^k$ combinations when $k \leq k_{\text{exact}}$.
- *GRASP repair* — $n_{\text{grasp\_repair}}$ randomised re-insertions for high iteration throughput.
- *Sub-MILP repair* — compact `gurobipy` model solved exactly for the k free aircraft; yields maximum quality per call.

The destroy strategy is selected either statically (e.g. `mixed`, `cluster`) or *adaptively* via **ALNS** (Adaptive LNS), which maintains per-strategy weights updated by a reward signal after each iteration.

4.2 Algorithm

Algorithm 2 Large Neighbourhood Search (LNS / ALNS)

if `warm_solution` provided **then**
  $s_0 \leftarrow$ `warm_solution` ▷ e.g. from TopologyHeuristic
else
  $s_0 \leftarrow \text{ConstructiveHeuristic}(t_{\text{init}})$
end if
$s^* \leftarrow \text{LocalSearch}(s_0,\ \texttt{max\_passes} = \max(5, \lfloor R/2 \rfloor))$
$s_{\text{cur}} \leftarrow s^*$
if destroy = `alns` **then**
  Initialise weights $w_d \leftarrow 1$ for each $d \in \mathcal{D}_{\text{alns}}$
end if
while time budget not exhausted **do**
  $k \sim \mathcal{U}[k_{\min}, k_{\max}]$
  $d^* \leftarrow$ selected destroy strategy (roulette if ALNS, else static)
  $\mathcal{R}_{\text{del}} \leftarrow \text{Destroy}(s_{\text{cur}}, k, d^*)$
  if $k \leq k_{\text{exact}}$ **then**
    $s \leftarrow \text{ExhaustiveRepair}(\mathcal{R}_{\text{del}})$
  else if gurobipy available **and** `repair` $\in$ {`submilp`, `auto`} **then**
    $s \leftarrow \text{SubMILPRepair}(\mathcal{R}_{\text{del}})$
    if $s = \emptyset$ **then** $s \leftarrow \text{GRASPRepair}(\mathcal{R}_{\text{del}})$
    end if
  else
    $s \leftarrow \text{GRASPRepair}(\mathcal{R}_{\text{del}})$
  end if
  $s_{\text{rb}} \leftarrow \text{Rebuild}(s)$ ▷ greedy schedule from position assignment
  if $z(s_{\text{rb}}) < z(s_{\text{cur}})$ **then** ▷ compare rebuild quality — not trial quality
    $s_{\text{cur}} \leftarrow s_{\text{rb}}$
    if $z(s_{\text{rb}}) < z(s^*)$ **then**
      $s^* \leftarrow \text{LocalSearch}(s_{\text{rb}})$ ▷ convergent polish on global best
      Update ALNS weight: global-best reward
    else
      Update ALNS weight: local-improvement reward
    end if
  else
    Update ALNS weight: no-improvement reward
  end if
  if no improvement for `restart_every` iterations **then**
    $s_{\text{cur}} \leftarrow \text{ConstructiveHeuristic}() + \text{LocalSearch}(\cdot,\ \texttt{max\_passes})$
  end if
end while
return s^*

Accept correctness. The repair step may produce a trial solution s whose objective is computed from an optimised schedule (GRASP or Gurobi). The actual schedule stored in s_{cur} is obtained by `Rebuild`, a greedy procedure that processes aircraft in earliest-start order. Because Rebuild may yield a different (potentially worse) objective than the trial, the accept condition uses *Rebuild quality*, not trial quality. Using the trial quality for acceptance but storing the Rebuild result would silently degrade s_{cur} over many iterations.

4.3 Initialisation

Two warm-start modes are supported:

1. **External warm solution.** If `warm_solution` is passed in `params`, it is used directly as the starting point. This allows, for example, the topology heuristic to seed the LNS with a high-quality initial solution rather than spending budget on an internal constructive run.
2. **Internal constructive run.** Otherwise, the LNS spends $t_{\text{init}} = \min(10\text{ s},\ 0.15 \cdot t_{\text{limit}})$ running the Constructive Heuristic (Section 3) with $\alpha = 0.7$.

After obtaining s_0, a **bounded local search** is applied with `max_passes` $= \max(5,\ \lfloor R/2 \rfloor)$. The cap prevents the initial polish from consuming the entire time budget on large instances (e.g. $R = 30$, where an unbounded convergent search can take tens of seconds). The same bound is applied to local searches triggered by restarts.

4.4 Destroy Strategies

The destroy operator selects k aircraft to remove from the current solution. Six strategies are available:

Strategy	Description
`worst`	Remove the k aircraft with the largest individual delays Δ_r.
`most_blocking`	Score each aircraft by the number of active blocking arcs it participates in; remove the k highest-scoring aircraft.
`random`	Uniform random selection.
`mixed`	Use `most_blocking` when total delay ≈ 0; otherwise use `worst`.
`cluster`	Space-time cluster destroy (see Section 4.4.2).
`mixed_cluster`	Use `most_blocking` when total delay ≈ 0; otherwise use `cluster`.

4.4.1 Adaptive LNS (`destroy = alns`)

When `destroy = alns`, the solver maintains a weight $w_d \geq 0$ for each strategy d in $\mathcal{D}_{\text{alns}} = \{$`worst`, `most_blocking`, `cluster`$\}$. At the start of each iteration, a strategy is sampled by roulette:

$$P(\text{select } d) = \frac{w_d}{\sum_{d'} w_{d'}} \tag{28}$$

After each iteration, the weight of the selected strategy is updated with exponential smoothing:

$$w_d \leftarrow (1 - \lambda)\, w_d \ + \ \lambda\, \rho \tag{29}$$

where $\lambda = 0.85$ is the learning rate and ρ is the reward:

Event	Reward ρ
Repair improved global best s^*	3.0
Repair improved current solution only	1.0
No improvement (rebuild $\geq s_{\text{cur}}$)	0.0

All weights are initialised to 1.0. Final weights are printed at the end of each run for diagnostics.

4.4.2 Space-Time Cluster Destroy (`cluster`)

The `cluster` strategy identifies a real bottleneck — a specific place and time where several aircraft compete for the same corridor — and destroys the entire involved neighbourhood simultaneously.

Algorithm 3 Space-Time Cluster Destroy

1: **Seed selection (roulette)**: weight r as $(\Delta_r + 1) \cdot (\mathrm{bs}(r) + 1)$; sample one aircraft r^*.
2: **Spatial cluster**: $\mathcal{P}^* \leftarrow \{p_{r^*}\} \cup \{p' : (p_{r^*}, p') \in \mathcal{B}\} \cup \{p' : (p', p_{r^*}) \in \mathcal{B}\}$
3: **Temporal filter** with margin $= 0.5 \cdot \bar{D}$: $\mathcal{C} \leftarrow \{r \in \mathcal{R} : p_r \in \mathcal{P}^* \ \wedge \ s_r < t_f^* + \text{margin} \ \wedge \ f_r > t_s^* - \text{margin}\}$
4: Rank $\mathcal{C}$ by $(\Delta_r + \mathrm{bs}(r))$ descending; take top k.
5: If $|\mathcal{C}| < k$, fill remainder with highest-Δ_r aircraft outside $\mathcal{C}$.
6: **return** k selected aircraft IDs.

4.5 Repair Operators

4.5.1 Exhaustive Repair ($k \leq k_{\text{exact}}$)

All $|\mathcal{P}|^k$ position combinations are enumerated. For each combination, a complete schedule is rebuilt (`_rebuild`) and the combination minimising the objective is selected. For default settings ($k_{\text{exact}} = 4$, $|\mathcal{P}| = 5$) this evaluates at most $5^4 = 625$ combinations per iteration.

4.5.2 GRASP Repair ($k > k_{\text{exact}}$, `repair = grasp`)

$n_{\text{grasp\_repair}}$ (default 12) random trials are performed. Each trial uses the GRASP construction from Section 3 to re-insert $\mathcal{R}_{\text{free}}$ into the partial schedule defined by $\mathcal{R}_{\text{fix}}$. The best trial is kept.

4.5.3 Sub-MILP Repair ($k > k_{\text{exact}}$, `repair = submilp / auto`)

When Gurobi is available, the sub-MILP repair replaces GRASP repair with an *exact* solve of a compact subproblem. A fresh `gurobipy` model is built containing only the k free aircraft; fixed aircraft appear as time-window constraints on position occupancy intervals. A module-level `gurobipy.Env` is created once at import time to amortise the $\sim$0.5 s startup cost.

Fallback monitoring. After each run, the solver prints the sub-MILP call statistics:

```
sub-MILP calls=N feasible=M (XX%) fallback_grasp=K
```

If the feasibility rate falls below 50%, a warning is emitted recommending a reduction of `k_max` or an increase of `repair_time_s`.

4.6 Schedule Rebuild

Given a position assignment $\{(r, p_r)\}_{r \in \mathcal{R}}$, `_rebuild` computes start times by processing aircraft in earliest-start order (or an explicit order if provided — used by the intra-position adjacent-swap in local search):

$$t_s(r) = \max\bigl(e_r,\ \text{pos\_free}[p] + \delta,\ t_s^{\text{block}}\bigr)$$

4.7 Local Search

Two invocations of the local search occur:

1. **Initial and restart polish:** bounded search with `max_passes` $= \max(5, \lfloor R/2 \rfloor)$. Prevents the first polish from consuming the entire time budget on large instances.

2. **Global-best polish:** convergent search (unbounded), fired only when a repair strictly improves the global best s^*. Repairs that only improve s_{cur} are accepted without local search, preserving iteration budget.

Both invocations use single-move and 2-opt swap operators (first-improvement).

4.8 Parameters

Parameter	Default	Description
`time_limit_s`	60	Total budget (seconds)
`min_separation`	10.0	Minimum gap between aircraft
`weight_makespan`	0.1	w_M
`weight_delay`	1.0	w_D
`weight_movements`	10	w_V
`k_min`	2	Minimum destroy size
`k_max`	8	Maximum destroy size
`k_exact`	4	Threshold for exhaustive vs. GRASP/sub-MILP repair
`n_grasp_repair`	12	GRASP repair trials when $k > k_{\text{exact}}$
`destroy`	`alns`	Destroy strategy
`repair`	`auto`	Repair: `grasp` | `submilp` | `auto`
`repair_time_s`	0.2	Gurobi time budget per sub-MILP call (s)
`restart_every`	50	Restart after N non-improving iterations
`warm_solution`	None	External solution injected as warm start
`seed`	None	Random seed

4.9 Complexity and Iteration Throughput

Repair mode	Cost per iteration	Typical iters / 60 s
Exhaustive ($k \leq k_{\text{exact}}$)	$O(\lvert\mathcal{P}\rvert^k)$ rebuilds	— (always used)
GRASP ($k > k_{\text{exact}}$)	$O(n_{\text{grasp}} \cdot R \cdot \lvert\mathcal{P}\rvert)$	∼200–400
Sub-MILP ($k > k_{\text{exact}}$)	Gurobi MIP, $\leq$ `repair_time_s`	∼100–300

5 Topology-Aware Heuristic

5.1 Motivation

The constructive heuristic assigns aircraft to positions without considering the long-term effect of the hangar topology on future flexibility. When a *heavy* aircraft (large D_r) occupies a *high-load* position (one that transitively blocks many others), it holds those downstream positions inaccessible for a long time, amplifying movements and delays in a cascade.

The topology-aware heuristic (`solvers/topology_heuristic.py`) augments the GRASP score with a *topology penalty* that discourages placing urgent/heavy aircraft in high-load positions.

5.2 Topological Pre-computation

Blocking load. For each position p, the *blocking load* $\ell(p)$ is the number of positions reachable from p via the transitive closure of $\mathcal{B}$:

$$\ell(p) = \left|\{p' \in \mathcal{P} : p \xrightarrow{*} p'\}\right|$$

Positions with $\ell(p) = 0$ block nobody (safe for heavy aircraft). High-ℓ positions amplify any congestion.

The computation is a DFS per position: $O(P^2)$.

Aircraft slack. The *slack* of aircraft r measures its scheduling margin:

$$\sigma_r = \max\bigl(0,\ T_r - e_r - D_r\bigr)$$

Small σ_r means the aircraft is *urgent*: it must start soon to avoid delay.

Urgency factor.

$$u_r = \frac{1}{1 + \sigma_r/\bar{\sigma}} \quad \in\ (0,\ 1] \qquad \bar{\sigma} = \frac{1}{R}\sum_r \sigma_r$$

$u_r \to 1$ for very urgent aircraft; $u_r \to 0$ for very relaxed ones.

5.3 Algorithm

Algorithm 4 Topology-Aware Heuristic (multi-start)

Pre-compute $\ell(p)$ for all p; compute σ_r, u_r, $\bar{\sigma}$ ▷ shared across starts
$s^* \leftarrow \emptyset$, $z^* \leftarrow \infty$
for $i = 0, \ldots,$ `n_starts` $- 1$ **do**
  seed$_i \leftarrow$ seed $+ i$; budget per start $\leftarrow t_{\text{limit}}/$`n_starts`
  while start time budget not exhausted **do**
    Add Gaussian noise $\epsilon \sim \mathcal{N}(0,\ \rho\bar{\sigma})$ to each σ_r ▷ $\rho =$ `noise_scale_ratio`
    Sort aircraft by $(\sigma_r + \epsilon \uparrow,\ D_r \downarrow)$ ▷ urgency order
    if LNS kick scheduled **then**
      Remove $k \in [\max(2, R/5),\ \max(3, R/3)]$ worst aircraft; re-insert
    else
      Construction: for each aircraft r in urgency order:
        Score: $\text{score}(r, p) = w_D\Delta_r + w_M t_f + w_V V' \cdot 2 + w_{\text{topo}} \cdot u_r \cdot \ell(p) \cdot w_D$
        $\alpha_{\text{eff}} = \alpha + u_r(1 - \alpha)$; GRASP-select position
    end if
    $s \leftarrow \text{LightLocalSearch}(s,\ \max(5, R),\ \text{deadline})$
    if $z(s) < z^*$ **then** $s^* \leftarrow s$; $z^* \leftarrow z(s)$
    end if
  end while
end for
return s^*

Sequential construction. Unlike the constructive heuristic (which evaluates all remaining aircraft × positions jointly), the topology heuristic processes aircraft one at a time in urgency order. For each aircraft it evaluates only the $|\mathcal{P}|$ positions and selects using biased-random GRASP. Processing in urgency order guarantees that the most constrained aircraft choose their position while the most desirable slots are still available.

5.4 Topology Penalty

The penalty added to the GRASP score when considering aircraft r at position p is:

$$\pi(r, p) = w_{\text{topo}} \cdot u_r \cdot \ell(p) \cdot w_D \tag{30}$$

Key properties:

- **Urgent aircraft** ($u_r \approx 1$): large penalty for high-load positions — they gravitate to $\ell(p) = 0$ slots.
- **Relaxed aircraft** ($u_r \approx 0$): penalty near zero — they can occupy any position without bias.
- **Zero-load positions** ($\ell(p) = 0$): penalty is always zero.

5.5 Adaptive GRASP Alpha

The GRASP randomness is adapted per aircraft:

$$\alpha_{\text{eff}}(r) = \alpha + u_r \cdot (1 - \alpha) \tag{31}$$

Recall that $\alpha = 1$ is **fully greedy** (only rank 0 selected) and $\alpha = 0$ is **uniform random** in the geometric weighting scheme $w_i = (1 - \alpha)^i$ (see Section 3). Urgent aircraft (high u_r) drive $\alpha_{\text{eff}} \to 1$, making selection nearly greedy — they pick the best available position with little randomness. Relaxed aircraft use $\alpha_{\text{eff}} \approx \alpha$, exploring more freely.

5.6 Construction Order and Noise Injection

Aircraft are processed in *urgency order*: smallest slack first, then largest duration among tied-slack aircraft. This ensures that the most constrained aircraft choose their positions while the most desirable slots are still available.

A small Gaussian noise $\epsilon \sim \mathcal{N}(0,\ 0.005\bar{\sigma})$ is added to each σ_r at the start of every iteration to diversify the ordering between iterations when aircraft have similar slack values.

5.7 LNS Perturbation Kicks

Every $\approx t_{\text{limit}}/8$ seconds (time-based, not iteration-count based) the heuristic fires a *perturbation kick* to escape topology-induced local optima:

1. Sort the current best solution by delay DESC, then finish time DESC. Take a pool of $\max(k \cdot 2,\ k+3)$ aircraft; shuffle the pool; select $k \in [\max(2, R/5),\ \max(3, R/3)]$ aircraft to remove.
2. Fix the remaining $R-k$ aircraft at their current positions; schedule them in urgency order.
3. Re-insert the k removed aircraft using topology-aware GRASP with $w_{\text{topo}} = 0$ (penalty disabled), allowing the heuristic to explore assignments that the penalty would normally prevent — including patterns where heavy aircraft occupy high-load positions with precise timing so no blocking materialises.

The time-based schedule ensures kicks fire with a consistent frequency regardless of instance size.

5.8 Bounded Local Search

The local search (`_light_local_search`) applies seven operators in priority order, bounded by `max_passes` $= \max(5, R)$ outer restarts. Candidate order is shuffled between passes to increase neighbourhood diversity. Any improvement restarts from operator 1.

Operators 1–3 (position assignment, $O(nP)$ per pass).

1. **Single-move**: try moving each aircraft to a different position; accept the first improvement.
2. **2-opt swap**: try swapping positions of every pair of aircraft with different positions; accept the first improvement.
3. **Intra-position adjacent swap**: for each position with ≥ 2 aircraft, try swapping consecutive pairs in earliest-start order; accept the first improvement.

Operators 4–7 (intra-position sequencing, $O(n^2P)$ per pass).

4. **Intra-position insertion**: for each position with ≥ 2 aircraft, try re-inserting each aircraft at every other slot in the same position's sequence; accept the first improvement.
5. **Non-adjacent intra-position swap**: extend Op 3 to all non-consecutive pairs within the same position.
6. **EDD repair**: for each position, evaluate three full-block reorderings — by earliest target finish (EDD), by slack ascending, and by delay/duration ratio descending — via `_rebuild`; accept the first improvement.
7. **Delay-block insertion**: identify the $K = \max(2, R/10)$ aircraft with the largest individual delay; attempt to move each earlier within its current position or reassign it to another position; accept the first improvement.

A *two-speed* mode selects the operator set based on budget per start: `ls_mode="fast"` (Ops 1–3 only) when budget per start is < 20 s, and `ls_mode="full"` (all operators) otherwise. This prevents the expensive operators from starving GRASP iterations in multi-start mode.

The `max_passes` counter decrements only when *no* operator finds an improvement. A deadline check after every operator prevents LS from overshooting the time budget (fixes a runaway issue observed at 133.9 s for a seed-8 run on R=20).

5.9 Parameters

Parameter	Default	Description
`time_limit_s`	60	Total budget (seconds)
`min_separation`	10.0	Minimum gap between aircraft
`weight_makespan`	0.1	w_M
`weight_delay`	1.0	w_D
`weight_movements`	10	w_V
`weight_topology`	1.0	w_{topo}
`alpha`	0.3	Base GRASP geometric decay
`kick_interval_ratio`	0.125	Kick interval as fraction of budget (0 = disabled)
`seed`	None	Random seed (base seed for multi-start)
`n_starts`	1	Number of independent restarts in the multi-start portfolio
`noise_scale_ratio`	0.005	Std. of Gaussian noise relative to mean slack $\bar{\sigma}$

5.10 Assignment Generator (`generate_assignments`)

Beyond its role as a complete solver, `TopologyHeuristic` exposes a lightweight interface for generating diverse position assignments without scheduling:

```
generate_assignments(instance_data, k=5, time_per_assign=2.0)
  -> list[dict[aircraft_id, position_id]]
```

The method runs k independent GRASP iterations, each with a budget of `time_per_assign` seconds and `ls_mode="fast"` (Ops 1–3 only), using seeds $\texttt{seed}, \texttt{seed}+1, \ldots, \texttt{seed}+k-1$. Each call returns only the position map $\{r \mapsto p\}$; timing is discarded.

Duplicate assignments (identical position maps across seeds) are removed before returning. Two assignments are considered distinct if at least one aircraft differs in its assigned position.

This interface is used by `TGRSolver` (Section 7) to generate candidate assignments that are then scheduled exactly by `FixedAssignmentScheduler` (Section 6).

Rationale. Fast GRASP without heavy LS produces weaker individual solutions but explores more diverse assignments within a fixed time budget. The scheduling quality of each assignment is then recovered by the exact FAS model rather than the greedy `_rebuild` scheduler.

5.11 Relationship to Other Methods and Ablation Variants

Configuration	Effect
$w_{\text{topo}} = 0$	Removes topology penalty; reduces to urgency-sorted GRASP with sequential (per-aircraft) position selection.
$\alpha = 1$	Fully greedy construction (only rank 0 selected); kicks provide the only diversification.
`kick_interval_ratio` $= 0$	Disables LNS perturbation kicks entirely; solver runs as pure iterated GRASP with no large-neighbourhood escape.

These three settings define the ablation study configurations (`topology_no_penalty`, `topology_greedy`, and `topology_no_kicks`) available in `run_experiments.py`.

6 Fixed-Assignment Scheduler

6.1 Motivation

The full MILP formulation (Section 2) optimises position assignment and timing jointly. When position assignments are already known — for example, from a topology heuristic run — all assignment variables become constants and the model reduces to a pure scheduling problem. Solving this reduced model is much faster: the dominant source of complexity in the full MILP (the position-assignment binary layer) is eliminated.

A second motivation is correctness. The Pyomo-based full MILP does not enforce the minimum separation δ between consecutive aircraft sharing the same position; it applies δ only in the blocking-arc constraints between different positions. The greedy scheduler (`_rebuild`) in the topology heuristic does enforce a δ-gap, but only via the `pos_free` counter, which can be inconsistent after local-search moves. The `FixedAssignmentScheduler` corrects both issues by making δ an explicit constraint.

6.2 Model

Let $\mathcal{R}$ be the set of aircraft, $p(r)$ the fixed position of aircraft r, e_r its earliest start, T_r its target finish, $D_r = \sum_{j \in \mathcal{J}_r} d_j$ its total processing time (sum of job durations along its precedence chain), and δ the minimum separation. Big-M is set to the sum of all D_r values plus the latest e_r.

Variables.

$$
\begin{aligned}
S_r, F_r &\geq 0 \quad \text{(start and finish of aircraft } r\text{)} \\
\mathit{delay}_r &\geq 0 \\
\mathit{makespan} &\geq 0 \\
q_{r,r'} &\in \{0,1\} \quad \forall\, r < r',\ p(r) = p(r') \quad \text{(ordering: 1 if } r \text{ precedes } r'\text{)} \\
u^{\text{in}}_{r,r'},\ u^{\text{out}}_{r,r'} &\in \{0,1\} \quad \forall \,\text{blocking pairs } (r, r')
\end{aligned}
$$

Job-level start times are not explicit variables. Because all jobs of each aircraft form a linear precedence chain with no branching, job starts are recovered deterministically from S_r after solving.

Constraints.

$$F_r = S_r + D_r \qquad \forall\, r \tag{32}$$

$$S_r \geq e_r \qquad \forall\, r \tag{33}$$

$$\mathit{delay}_r \geq F_r - T_r \qquad \forall\, r \tag{34}$$

$$\mathit{makespan} \geq F_r \qquad \forall\, r \tag{35}$$

Same-position ordering with δ separation (for each pair $r < r'$ with $p(r) = p(r')$):

$$S_{r'} \geq F_r + \delta - M(1 - q_{r,r'}) \tag{36}$$

$$S_r \geq F_{r'} + \delta - M\, q_{r,r'} \tag{37}$$

Blocking arcs (for each pair (r, r') whose fixed positions form a blocking arc $p(r) \to p(r')$ in the hangar topology):

$$S_{r'} \geq F_r - M\, u^{\text{in}}_{r,r'} \tag{38}$$

$$S_r \geq F_{r'} - M(1 - u^{\text{in}}_{r,r'}) \tag{39}$$

$$S_r \geq F_{r'} - M\, u^{\text{out}}_{r,r'} \tag{40}$$

$$S_{r'} \geq F_r - M(1 - u^{\text{out}}_{r,r'}) \tag{41}$$

These four constraints enforce that, for each blocking arc, either r finishes before r' starts (entry direction) or r' finishes before r starts (exit direction).

Objective.

$$\min \; w_M \cdot \mathit{makespan} + w_D \cdot \sum_r \mathit{delay}_r + w_V \cdot V \tag{42}$$

where V (the number of blocking movements) is a constant for a fixed assignment, computed before model construction.

6.3 Implementation

The model is implemented in `solvers/fixed_assignment_scheduler.py` as class `FixedAssignmentScheduler`, built directly with the `gurobipy` API (not via Pyomo). This eliminates the Pyomo model construction overhead, which was measured at $\sim$110 s for $R = 30$; the gurobipy FAS builds in < 0.1 s for all instance sizes tested.

Job chain recovery. After solving, job start times are reconstructed from S_r by iterating the precedence chain in topological order:

$$S_{j_1} = S_r, \quad S_{j_{k+1}} = S_{j_k} + d_{j_k}.$$

Interface.

```
fas = FixedAssignmentScheduler()
fas.configure(time_limit_s=60, min_separation=10, ...)
solution = fas.solve(instance_data, assignment)
# assignment: {aircraft_id: position_id}
# solution: standard dict compatible with check_solution.py
```

The solution dict includes `_build_time_s` and `_solve_time_s` for timing diagnostics.

6.4 Parameters

Parameter	Default	Description
`time_limit_s`	60	Gurobi time limit (seconds)
`min_separation`	10.0	δ: minimum gap between consecutive aircraft at the same position
`weight_makespan`	0.1	w_M
`weight_delay`	1.0	w_D
`weight_movements`	10	w_V
`mip_gap`	0.0	Gurobi MIPGap

6.5 Known Modelling Difference vs Topology Heuristic

FAS aggregates each aircraft to a single block of duration $D_r = \sum_j d_j$. The blocking-arc constraints (38)–(41) therefore use $F_r = S_r + D_r$ as the aircraft's finish time when determining whether a blocking event occurs. This is conservative: if only the first job of r creates the spatial conflict, the actual blocking window is shorter than D_r.

The topology heuristic's `_rebuild` function schedules job-by-job and evaluates blocking at each job's start/finish, so it implicitly uses job-level timing for blocking arc decisions.

Consequence (observed in Experiment 03-A): on dense instances with many blocking arcs (e.g. $R = 20$), FAS produces a worse objective than `topology_ms6` on the *same* assignment.

Both solutions pass RQ08 (verified 2026-05-14: topology min gap = 10.00 on all instances). The gap is not a feasibility bug but a modelling conservatism: FAS has a strictly smaller feasible region for the timing problem.

A future improvement is to pass job-level timing to FAS and compute blocking windows per-job rather than per-aircraft.

6.6 Model Size and Build Time

For a fixed assignment with R aircraft and P positions:

- **Continuous variables**: $3R + 1$ (S_r, F_r, $delay_r$, makespan).
- **Binary variables**: $\binom{R_{\text{same}}}{2}$ ordering binaries (where R_{same} is the number of aircraft sharing a position) plus $2|\mathcal{B}|$ blocking binaries ($\mathcal{B}$ = set of blocking arc pairs).
- **Build time**: < 0.02 s for all instances tested ($R \leq 30$), compared to $\sim$110 s for the Pyomo full MILP at $R = 30$.

7 TGR-MILP Solver

7.1 Motivation

The topology heuristic (Section 5) produces good position assignments quickly but uses a greedy scheduler (`_rebuild`) to determine aircraft timing. The Fixed-Assignment Scheduler (Section 6) can optimise timing exactly for any given assignment, but relies on the quality of that assignment.

`TGRSolver` (Topology-Guided Restricted MILP) combines both: it generates K diverse position assignments via fast GRASP and schedules each one exactly with the FAS, returning the best result.

7.2 Algorithm

Algorithm 5 TGRSolver

Require: instance data, K, t_{topo}, t_{MILP}

$\mathcal{A} \leftarrow$ `TopologyHeuristic.generate_assignments`$(K,\ t_{\text{topo}}/K)$
$\mathcal{S} \leftarrow \emptyset$
for each assignment $a \in \mathcal{A}$ **do**
    $s \leftarrow$ `FixedAssignmentScheduler.solve`$(a,\ t_{\text{MILP}})$
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{s\}$
end forreturn $\arg\min_{s \in \mathcal{S}} \text{obj}(s)$

Time budget split. With a total budget T:

$$t_{\text{topo}} + K \cdot t_{\text{MILP}} \leq T.$$

The default configurations tested are:

- `tgr_k5`: $K = 5$, $t_{\text{topo}} = 10\,\text{s}$ (2 s per assignment), $t_{\text{MILP}} = 10\,\text{s}$ per FAS. Total = 60 s.
- `tgr_k3`: $K = 3$, $t_{\text{topo}} = 9\,\text{s}$ (3 s per assignment), $t_{\text{MILP}} \approx 17\,\text{s}$ per FAS. Total = 60 s.

7.3 Implementation

Implemented in `solvers/tgr_solver.py` as class `TGRSolver`. The solver is compatible with the `Application` interface via `configure_solver(**kwargs)`.

Deduplication. Assignments are deduplicated before scheduling: two assignments are identical if every aircraft maps to the same position. Only unique assignments are passed to the FAS, so the effective number of MILP solves may be less than K.

Best-solution selection. The solution with the lowest objective value across all FAS runs is returned. Infeasible or error runs are excluded.

7.4 Observed Performance

Results from Experiment Series 02 (Section 9.4) show that `tgr_k5` matches or beats the MILP baseline for small instances ($R \leq 5$) in under 10 s. For medium/large instances ($R \geq 20$), TGR underperforms `topology_ms6` because the 2 s per-assignment GRASP (with `ls_mode="fast"`) generates weaker assignments than a full 60 s topology run with local search.

The corrective approach — running `topology_ms6` for the full budget and passing its best assignment to FAS — is evaluated as `fas_on_topo` in Experiment Series 03 (Section 10.3).

7.5 Parameters

Parameter	Default	Description
`n_assignments`	5	K: number of assignments to generate
`time_topology_s`	10.0	Total topology budget (shared across K runs)
`time_per_milp_s`	10.0	FAS time limit per assignment
`min_separation`	10.0	δ passed to FAS
`weight_makespan`	0.1	w_M
`weight_delay`	1.0	w_D
`weight_movements`	10	w_V
`seed`	1	Base seed for topology GRASP

8 Experiment Series 01: Topology Calibration and TGR-MILP

8.1 Motivation and Research Questions

The multi-start topology portfolio (`topology_msN`) experiment of 2026-05-13 revealed two persistent gaps:

1. **Regression on small instances (R=10).** `topology_ms3/ms6/ms12` returned objectives of 16.93–24.03 against the MILP optimum of 4.79. With a 60 s budget split into N slices, each slice is too short (5–20 s) for the O(n^2P) operators (Ops 4–7) to complete even one GRASP iteration.
2. **Large gap on medium instances (R=20).** The time-limited MILP achieved 145.40 vs topology ≈233. Both solutions use zero movements; the gap lies entirely in delay, suggesting the MILP finds better *timing/sequencing* within the same positional assignment rather than fundamentally different position choices.

This series addresses both gaps through two strategies:

- **Topology calibration** (Exp. 01-A): two-speed local search and size-adaptive multi-start to fix the R=10 regression.
- $|C_r| = 1$ **fix-position diagnostic** (Exp. 01-B): fix all position assignments from the best topology solution and let the MILP optimise only timing/sequencing variables. This isolates whether the gap is assignment-based or timing-based.

8.2 Implementation

Topology calibration. A *two-speed* local search distinguishes cheap operators (Ops 1–3, $O(nP)$ per pass) from expensive ones (Ops 4–7, $O(n^2P)$ per pass). When budget per start < 20 s, only Ops 1–3 are used (`ls_mode="fast"`); otherwise all operators are active (`ls_mode="full"`). A size-adaptive guard also reduces `n_starts` to 1 for instances with $R \leq 10$, ensuring a single full-budget run with `ls_mode="full"`.

Fix-position diagnostic (`milp_fix1`). All Pyomo binary position variables `vAircraftPosition`$[r, p]$ are fixed to 1 if p equals the position assigned by `topology_ms6`, and to 0 otherwise, before calling Gurobi. This reduces the MILP to a continuous optimisation (plus sequencing binaries) over a single positional assignment, isolating the timing gap.

8.3 Results

Log file. `run_experiments_20260514_073120.log`

Configuration. Shared: $w_M = 0.1$, $w_D = 1.0$, $w_V = 10$, $t_{\text{limit}} = 60$ s, MIPGap=0, $\alpha = 0.3$, $w_{\text{topo}} = 1$, seed=1.

8.3.1 Exp. 01-A — Topology Calibration (two-speed LS + size-adaptive starts)

Observation — R=10 regression persists partially. The size-adaptive guard correctly reduces `n_starts` to 1 for $R \leq 10$, but `topology_ms6` in full-budget mode now gives 11.68 (vs 24.03 in the previous run), a significant improvement. The remaining gap of 11.68 vs 4.79 reflects sub-optimal position assignments (confirmed by Exp. 01-B below).

8.3.2 Exp. 01-B — $|C_r| = 1$ Fix-Position Diagnostic (`milp_fix1`)

* R=30 timing exceeded the 60 s Gurobi limit due to Python/Pyomo overhead; see discussion.

Table 1: Topology calibration results (2026-05-14). Bold = best heuristic per instance.

Instance	Method	Status	Obj	Makespan	Delay	Mov	Time(s)
`scn_custom` (R=10)	milp_baseline	optimal	**4.79**	47.90	0.00	0	16.2
	topology_ms3	heuristic	17.10	72.72	9.83	0	60.0
	topology_ms6	heuristic	**11.68**	62.59	5.42	0	60.0
	topology_ms12	heuristic	24.03	73.37	16.69	0	60.0
`scn_few-loose` (R=3)	milp_baseline	optimal	**3.12**	31.22	0.00	0	0.2
	topology_ms3	heuristic	**3.12**	31.22	0.00	0	60.0
	topology_ms6	heuristic	**3.12**	31.22	0.00	0	60.0
	topology_ms12	heuristic	**3.12**	31.22	0.00	0	60.0
`scn_few-tight` (R=2)	milp_baseline	optimal	**6.39**	63.88	0.00	0	0.3
	topology_ms3	heuristic	**6.39**	63.88	0.00	0	60.0
	topology_ms6	heuristic	**6.39**	63.88	0.00	0	60.0
	topology_ms12	heuristic	**6.39**	63.88	0.00	0	60.0
`scn_heavy-tight` (R=30)	milp_baseline	aborted	—	—	—	—	—
	topology_ms3	heuristic	**231.49**	239.95	187.49	2	61.0
	topology_ms6	heuristic	250.67	241.40	206.53	2	60.0
	topology_ms12	heuristic	250.48	244.18	226.06	0	60.2
`scn_many-medium` (R=20)	milp_baseline	timeLim	**200.78**	105.40	190.24	0	69.1
	topology_ms3	heuristic	**216.62**	111.01	205.52	0	60.0
	topology_ms6	heuristic	242.84	107.68	232.07	0	60.0
	topology_ms12	heuristic	233.09	112.19	221.87	0	60.0

8.4 Analysis

The gap is almost entirely in timing, not position assignment. The $|C_r| = 1$ diagnostic is unambiguous: fixing position assignments from the topology solution and letting the MILP optimise timing alone dramatically closes the gap.

- **R=20**: `milp_fix1` = 43.35 vs `milp_baseline` = 200.78 (best known under time limit). The topology chooses excellent positions; the heuristic local search simply cannot replicate the exact intra-position sequencing that Gurobi finds.
- **R=30**: `milp_fix1` = 19.39 vs topology best = 231.49. An order-of-magnitude improvement by fixing positions. The topology is finding good spatial assignments but the heuristic scheduler is far from optimal in timing.
- **R=10**: `milp_fix1` = 5.53 vs optimal = 4.79 (gap of 0.74). Small residual assignment gap — topology_ms6 chose slightly sub-optimal positions for one aircraft.
- **R≤5**: `milp_fix1` = `milp_baseline` — optimal in all cases.

Implication: TGR-MILP is the correct next step. The evidence strongly supports the Topology-Guided Restricted MILP strategy:

1. Run `topology_ms6` for a fraction of the budget to get position candidates.
2. Fix (or restrict) position variables in the MILP.
3. Let Gurobi optimise timing with the vastly reduced search space.

R=30 timing issue. `milp_fix1` on R=30 ran for 171.8 s total, exceeding the 60 s Gurobi `TimeLimit`. The excess is Python/Pyomo overhead: model instantiation, variable fixing, and

Table 2: Fix-position diagnostic: MILP with all positions fixed from `topology_ms6`.

Instance (R)	Method	Obj	Makespan	Delay	Mov	Time(s)
scn_custom (R=10)	milp_baseline	4.79	47.90	0.00	0	16.2
	topology_ms6	11.68	62.59	5.42	0	60.0
	milp_fix1	**5.53**	55.25	0.00	0	**1.1**
scn_few-loose (R=3)	milp_baseline	3.12	31.22	0.00	0	0.2
	topology_ms6	3.12	31.22	0.00	0	60.0
	milp_fix1	**3.12**	31.22	0.00	0	0.2
scn_few-tight (R=2)	milp_baseline	6.39	63.88	0.00	0	0.3
	topology_ms6	6.39	63.88	0.00	0	60.0
	milp_fix1	**6.39**	63.88	0.00	0	0.4
scn_heavy-tight (R=30)	milp_baseline	aborted	—	—	—	—
	topology_ms6	250.67	241.40	206.53	2	60.0
	milp_fix1	**19.39**	186.57	0.74	0	171.8*
scn_many-medium (R=20)	milp_baseline	200.78	105.40	190.24	0	69.1
	topology_ms6	242.84	107.68	232.07	0	60.0
	milp_fix1	**43.35**	77.68	35.58	0	21.2

solution extraction for a 30-aircraft instance with dense blocking arcs require significant preprocessing time. The Gurobi solve itself respected the limit; the remaining ≈110 s is model setup. This overhead must be accounted for in the TGR-MILP budget allocation: for R=30, the effective Gurobi budget will be $60 - t_{\text{setup}}$ s.

Topology regression on R=10 — partially resolved. The two-speed LS and size-adaptive guard recover `topology_ms6` from 24.03 (previous run) to 11.68. The remaining gap of 11.68 vs 4.79 is a position-assignment error (one aircraft misplaced), not a timing error. Improving the GRASP scoring or restricting the candidate set for small instances would address this. The TGR-MILP pipeline naturally fixes this: even with one misplaced aircraft, `milp_fix1` reaches 5.53 in 1.1 s.

8.5 Next Steps

Based on these results, the planned TGR-MILP pipeline is validated. Next experiments:

1. **Exp. 01-C**: Implement the split-budget TGR-MILP: topology_ms6 for t_1 seconds, then MILP with fixed positions for $60 - t_1 - t_{\text{setup}}$ seconds. Tune t_1 for R=20 and R=30.
2. **Exp. 01-D**: Relax $|C_r| = 1$ to $|C_r| = 2$ (top-2 topology positions per aircraft) to allow the MILP to correct single-aircraft position errors observed in R=10.
3. **Investigate setup overhead**: profile Pyomo model instantiation for R=30 and consider caching the compiled model to reduce the 110 s penalty.

9 Experiment Series 02: TGR-MILP Pipeline

9.1 Motivation

The $|C_r| = 1$ diagnostic of Series 01 confirmed that the gap between the topology heuristic and the MILP is dominated by timing/sequencing, not position assignment. This motivates a paradigm shift:

> *Use the topology heuristic as a fast spatial-assignment generator; use a scheduling MILP to optimise timing exactly within those fixed assignments.*

This section describes the implementation of this pipeline and its experimental validation.

9.2 Phase 0 — Audit of `milp_fix1` Results

Before proceeding, the `milp_fix1` results were audited for feasibility consistency between the Pyomo MILP and the heuristic scheduler.

Finding: minimum separation δ absent from same-position non-overlap. Reading `models/milp_pyomo.py`, the non-overlap constraints at the job level (`fcNonOverlapOrder1/2`) enforce only:

$$S_{j'} \geq F_j \quad \text{when } j \text{ precedes } j' \text{ at the same position}$$

with no $+\delta$ term. The parameter `pMinSeparation` (δ) appears exclusively in the *blocking-arc* constraints (`fcBetaInUB/LB`), which govern entry/exit movements between different positions — not the gap between consecutive aircraft sharing the same position.

By contrast, the heuristic (`_find_best_start` in `constructive_heuristic.py`) enforces:

$$S_r \geq \texttt{pos\_free}[p] + \delta \quad \text{(when position } p \text{ is already occupied)}$$

with $\delta = 10$ minutes by default.

Consequence. The Pyomo MILP packs consecutive aircraft at the same position with zero gap; the heuristic requires a 10-minute gap. Measured on the `milp_fix1` solutions: every consecutive same-position pair has a gap of exactly 0.00 min. The checker (`check_solution.py`) only verifies non-overlap ($S_{j'} \geq F_j$ up to tolerance), so it reports all solutions as `compliant=True` despite the inconsistency.

Implication. The `milp_fix1` objectives (5.53 for R=10, 43.35 for R=20, 19.39 for R=30) are real improvements over the topology heuristic, but they are obtained under a looser feasibility definition. A fair comparison requires the scheduling MILP to include the same δ constraint.

This observation motivated the design of `FixedAssignmentScheduler` (`solvers/fixed_assignment_schedu`) which explicitly adds $+\delta$ to same-position ordering constraints.

Additional check: `check_solution` passes all RQ01–RQ07. All three `milp_fix1` solutions (R=10, 20, 30) passed all existing requirements checks. The minimum-separation gap is a specification deficiency in the checker, not a model infeasibility.

9.3 Implementation

9.3.1 `FixedAssignmentScheduler` (`solvers/fixed_assignment_scheduler.py`)

A `gurobipy`-native model that solves the scheduling problem with fixed position assignments. Key differences from the full Pyomo MILP:

- **Removed variables**: `vAircraftPosition`, `vJobPosition`, `vJobOrder` — replaced by same-position ordering binaries $q_{r,r'} \in \{0,1\}$ (one per pair of aircraft at the same position).
- **Aircraft aggregation**: each aircraft is treated as a single block with $D_r = \sum_j d_j$ (valid because all job precedences are intra-aircraft chains with no alternatives). Job start times are recovered deterministically from S_r after solving.
- **Correct δ enforcement**: same-position ordering constraints include $+\delta$:

$$S_{r'} \geq F_r + \delta - M(1 - q_{r,r'}) \quad \text{and} \quad S_r \geq F_{r'} + \delta - M\, q_{r,r'}$$

- **Blocking variables retained**: $u^{\text{in}}_{r,r',p,p'}$ and $u^{\text{out}}_{r,r',p,p'}$ are kept for pairs whose fixed positions form a blocking arc. Position guard terms collapse to constants.
- **Build time**: < 0.1 s for R=10, < 0.5 s for R=20, measured empirically (vs $\sim$110 s Pyomo overhead for R=30).

9.3.2 `generate_assignments` in `TopologyHeuristic`

A new method `generate_assignments(k, time_per_assign)` runs k fast GRASP iterations (no local search) to generate diverse position assignments. Each call returns a dict {aircraft_id: position_id}. Duplicate assignments (identical position maps) are deduplicated before returning.

9.3.3 `TGRSolver` (`solvers/tgr_solver.py`)

The full pipeline:

1. Generate K diverse assignments via `generate_assignments`.
2. For each assignment, solve `FixedAssignmentScheduler` with a per-MILP time budget.
3. Return the schedule with the lowest objective.

Two configurations are tested:

- `tgr_k5`: $K = 5$, 10 s topology budget (2 s per assignment), remaining $\approx$50 s split across 5 MILPs ($\approx$10 s each).
- `tgr_k3`: $K = 3$, 9 s topology budget (3 s per assignment), remaining $\approx$51 s split across 3 MILPs ($\approx$17 s each).

9.4 Results

Log file. `run_experiments_20260514_*.log`

Configuration. Shared: $w_M = 0.1$, $w_D = 1.0$, $w_V = 10$, $t_{\text{limit}} = 60$ s, MIPGap=0, $\alpha = 0.3$, $w_{\text{topo}} = 1$, seed=1.

9.4.1 TGR-MILP vs baselines

Log file. `run_experiments_20260514_085415.log`
Build times (FAS model): < 0.02 s per assignment for all instances, confirming sub-second overhead vs Pyomo's $\sim$110 s.

9.5 Analysis

TGR underperforms topology_ms6 on R=20/30. The pipeline underperforms for the two largest instance sizes. The root cause is not the scheduler but the quality of input assignments:

Table 3: TGR-MILP pipeline vs baselines (2026-05-14). Bold = best per instance.

Instance (R)	Method	Obj	Makespan	Delay	Mov	Time(s)
`scn_custom` (R=10)	milp_baseline	**4.79**	47.90	0.00	0	23.8
	topology_ms6	7.06	65.25	0.54	0	60.0
	tgr_k5	16.93	62.59	10.67	0	10.0
	tgr_k3	16.93	62.59	10.67	0	9.0
`scn_few-loose` (R=3)	milp_baseline	**3.12**	31.22	0.00	0	0.2
	topology_ms6	**3.12**	31.22	0.00	0	60.0
	tgr_k5	**3.12**	31.22	0.00	0	10.0
	tgr_k3	**3.12**	31.22	0.00	0	9.0
`scn_few-tight` (R=2)	milp_baseline	**6.39**	63.88	0.00	0	0.2
	topology_ms6	**6.39**	63.88	0.00	0	60.0
	tgr_k5	**6.39**	63.88	0.00	0	10.0
	tgr_k3	**6.39**	63.88	0.00	0	9.0
`scn_heavy-tight` (R=30)	milp_baseline	aborted	—	—	—	—
	topology_ms6	**217.47**	246.29	192.85	0	64.8
	tgr_k5	288.17	261.04	262.07	0	60.2
	tgr_k3	305.72	263.85	279.33	0	60.4
`scn_many-medium` (R=20)	milp_baseline	**145.40**	96.21	135.78	0	67.9
	topology_ms6	249.68	107.33	238.95	0	60.0
	tgr_k5	267.78	105.69	257.21	0	60.1
	tgr_k3	267.78	105.69	257.21	0	58.5

- With 10 s of topology budget split across 5 assignments, each GRASP run has only 2 s and uses `ls_mode="fast"` (no intra-position operators). The resulting assignments are weaker than those produced by `topology_ms6` with 60 s full local search.
- The `FixedAssignmentScheduler` with correct $\delta = 10$ cannot recover the scheduling quality that the heuristic achieves by co-optimising assignment and timing simultaneously over 60 s.
- For R=20, tgr_k5 gives 267.78 vs topology_ms6 at 249.68: the FAS MILP is optimal *given* the assignment, but the assignment itself is worse than what the topology heuristic finds with a full budget.

The $\delta = 10$ constraint is the decisive factor. Comparing `milp_fix1` (no δ on same-position pairs, R=20: 43.35) with `tgr_k5` (correct δ, R=20: 267.78) shows that the $\delta = 10$ constraint drastically changes the problem. The milp_fix1 result was not a fair benchmark.

With correct δ, the scheduling problem is much harder: consecutive aircraft must be separated by 10 minutes, which for dense instances (R=30, 4 positions, many aircraft per position) leaves little flexibility in timing.

FAS overhead: confirmed sub-second. Build times are < 0.02 s per assignment for all instances. The Pyomo overhead of $\sim$110 s for R=30 was entirely a model-construction artefact, not a Gurobi limitation.

Small instances: TGR matches or equals optimal in 10 s. For R$\leq$5, TGR matches the MILP optimal in under 10 total seconds — 7–8$\times$ faster than the 60 s MILP. This is the expected regime for TGR: small instances where each GRASP + FAS cycle is fast.

9.6 Root Cause and Corrective Experiment

The separation between assignment generation and scheduling is sound in principle, but the assignment generator must be as good as possible because the FAS MILP cannot improve the assignment, only the timing.

Key insight: for medium/large instances, the topology heuristic with full local search (`topology_ms6`) already produces the best achievable assignments within a 60 s budget. The correct pipeline is therefore:

1. Run `topology_ms6` for the full 60 s budget.
2. Pass the best solution's assignment to `FixedAssignmentScheduler` with a *short additional budget* (or zero extra budget, checking whether FAS improves over the heuristic timing within the same assignment).

This is effectively "honest milp_fix1" — the same idea as Exp. 01-B but with the correct δ constraint. The experiment `fas_on_topo` in Series 03 tests this directly.

A second corrective direction is to increase the per-assignment topology budget (e.g. 20 s per assignment for $K = 3$), so assignments are stronger before FAS scheduling.

9.7 Next Steps

1. **Exp. 03-A (`fas_on_topo`)**: *done* — see Section 10.
2. **Exp. 03-B (`tgr_long`)**: increase topology budget to 20 s per assignment with $K = 3$, giving each GRASP run 20 s with `ls_mode="full"`. Test whether stronger assignments recover TGR performance.
3. **Fix `check_solution.py`**: add RQ08 to verify consecutive aircraft at the same position are separated by $\geq \delta$.
4. $|C_r| = 2$: after confirming Exp. 03-A results, add flexibility for the 1–2 aircraft where topology chose a sub-optimal position.

10 Experiment Series 03: FAS on Topology Assignment

10.1 Motivation

Series 02 showed that `TGRSolver` underperforms `topology_ms6` on medium/large instances because fast GRASP (2 s, no LS) generates weaker assignments than a full-budget topology run. The corrective hypothesis is:

> *If the FAS is given the best assignment that `topology_ms6` can find in 60 s, can the exact scheduler improve on the heuristic's timing within that same assignment?*

This is "honest `milp_fix1`": the same separation as Exp. 01-B but with the correct $\delta = 10$ constraint in the FAS model.

10.2 Configuration

- **Experiment label**: `fas_on_topo`
- **Mechanism**: `topology_ms6` runs first (60 s); its best solution's position map is extracted from the warm cache and passed to `FixedAssignmentScheduler`. FAS time limit = 60 s.
- Shared parameters: $w_M = 0.1$, $w_D = 1.0$, $w_V = 10$, $\delta = 10$, MIPGap=0, seed=1.

10.3 Results

Log file. `run_experiments_20260514_093456.log`

Table 4: FAS on topology_ms6 assignment vs baselines (2026-05-14). Bold = best per instance.

Instance (R)	Method	Obj	Makespan	Delay	Mov	Time(s)
`scn_custom` (R=10)	milp_baseline	**4.79**	47.90	0.00	0	21.1
	topology_ms6	17.44	67.02	10.74	0	60.0
	tgr_k5	16.93	62.59	10.67	0	10.0
	fas_on_topo	**7.06***	65.25	0.54	0	0.0
`scn_few-loose` (R=3)	milp_baseline	**3.12**	31.22	0.00	0	0.1
	topology_ms6	**3.12**	31.22	0.00	0	60.0
	tgr_k5	**3.12**	31.22	0.00	0	10.0
	fas_on_topo	**3.12**	31.22	0.00	0	0.0
`scn_few-tight` (R=2)	milp_baseline	**6.39**	63.88	0.00	0	0.6
	topology_ms6	**6.39**	63.88	0.00	0	60.0
	tgr_k5	**6.39**	63.88	0.00	0	10.0
	fas_on_topo	**6.39**	63.88	0.00	0	0.0
`scn_heavy-tight` (R=30)	milp_baseline	aborted	—	—	—	—
	topology_ms6	**256.45**	244.46	232.00	0	67.8
	tgr_k5	333.36	251.42	288.22	2	60.1
	fas_on_topo	227.57†	239.73	203.60	0	60.3
`scn_many-medium` (R=20)	milp_baseline	**200.78**	105.40	190.24	0	70.8
	topology_ms6	242.86	111.01	231.76	0	60.0
	tgr_k5	242.65	106.21	232.03	0	60.1
	fas_on_topo	253.59	111.93	242.40	0	20.4

* `fas_on_topo` elapsed < 0.1 s for R=10 (optimal, small model); † time-limited at 60 s for R=30 (FAS not proven optimal).

10.4 Analysis

R=10: large improvement over topology_ms6. `fas_on_topo` reduces the objective from 17.44 to 7.06 on the R=10 instance — a 59.5% improvement — while using the *same* position assignment. This confirms that the heuristic scheduler (`_rebuild`) leaves significant timing slack unexploited. The residual gap vs the `milp_baseline` (4.79) is due to the position assignment itself, not the scheduler.

R=30: FAS beats topology_ms6 under time limit. `fas_on_topo` achieves 227.57 vs topology_ms6's 256.45, a 11.3% improvement, even though Gurobi did not prove optimality within 60 s (status: `maxTimeLim`). The FAS model on R=30 is substantially harder than on R=10 because $\delta = 10$ with 30 aircraft and 5 positions creates dense sequencing constraints. More time would likely yield a better or proven-optimal solution.

R=20: FAS *worse* than topology_ms6. `fas_on_topo` gives 253.59 vs 242.86 for topology_ms6 — a 4.4% regression. Gurobi solved to optimality in 20 s, so the FAS optimum is exact given the topology_ms6 assignment. This means: the optimal schedule for that assignment is 253.59, yet the greedy heuristic on the *same* assignment achieves 242.86.

Root cause of R=20 regression. The greedy heuristic (`_rebuild`) in `topology_ms6` does *not* enforce $\delta = 10$ between consecutive aircraft at the same position. It uses `pos_free`$[p] + \delta$ only when the position is currently occupied — but for the initial schedule or after improvements, some consecutive pairs may be packed tighter or later without the δ gap. Conversely, the FAS model enforces δ strictly via the ordering constraints. For dense instances (R=20, many aircraft per position), this makes the FAS feasible region strictly smaller than the heuristic's, so the FAS optimum can be worse than the heuristic objective which "cheats" on the separation.

Implication. The R=20 regression is not a model bug — it reflects that `topology_ms6` is producing schedules that violate the $\delta = 10$ same-position separation in the strict sense. Fixing this in the heuristic (enforcing δ consistently) would likely raise the topology_ms6 objective on R=20 and make FAS competitive. This is the next modelling step.

Small instances: FAS is the right tool. For R≤5, FAS matches optimal in < 0.1 s, vs 60 s MILP baseline and 60 s topology. FAS is the clear winner in speed for small instances.

10.5 Next Steps

1. **Enforce δ consistently in heuristic**: audit `_rebuild` and `_find_best_start` to ensure δ is always added between consecutive aircraft at the same position, not just when the position is currently occupied.
2. **Add RQ08 to `check_solution.py`**: verify that consecutive aircraft at the same position satisfy $S_{r'} \geq F_r + \delta$.
3. **Re-run after heuristic fix**: compare topology_ms6 vs fas_on_topo on R=20 once the heuristic uses consistent δ.
4. **Exp. 03-B (`tgr_long`)**: test TGR with 20 s per assignment ($K = 3$, `ls_mode="full"`) to see if stronger assignments close the gap vs topology_ms6.

11 Experiment Series 04: Strict-δ Baselines

11.1 Motivation

All results in Series 01–03 used an inconsistent feasibility definition: the Pyomo MILP did not enforce the minimum separation $\delta = 10\,\text{min}$ between consecutive aircraft at the same position, while the topology heuristic and FAS did. This made direct objective comparisons invalid.

Two fixes were applied on 2026-05-14:

1. **RQ08 added to `check_solution.py`**: verifies $S_{r'} \geq F_r + \delta$ for every consecutive same-position pair. Audit confirmed `topology_ms6` passes RQ08 on all instances (min gap = 10.00 exactly). The MILP baseline (pre-fix) would have violated RQ08.
2. **MILP Pyomo corrected**: two new constraints added to `models/milp_pyomo.py` (see equations (15) and (16) in Section 2):

$$S_{r'} \geq F_r + \delta - M(1 - q_{rr'p}) - M(1 - x_{rp}) - M(1 - x_{r'p})$$
$$S_r \geq F_{r'} + \delta - M(1 - q_{r'rp}) - M(1 - x_{rp}) - M(1 - x_{r'p})$$

   The model size increases by $2|\mathcal{A}^2 \times \mathcal{P}|$ rows (e.g. 14326 → 14776 for $R = 10$, 45271 → 47171 for $R = 20$).

11.2 Configuration

Shared: $w_M = 0.1$, $w_D = 1.0$, $w_V = 10$, $t_{\text{limit}} = 60\,\text{s}$, MIPGap=0, $\delta = 10$, $\alpha = 0.3$, $w_{\text{topo}} = 1$, seed=1.

All four methods (`milp_baseline`, `topology_ms6`, `tgr_k5`, `fas_on_topo`) now operate under the same feasibility definition.

11.3 Results

Log file. `run_experiments_20260514_110159.log`
*`fas_on_topo` elapsed $< 0.1\,\text{s}$ for R=10 (optimal, small model). †time-limited at 60 s (not proven optimal).

11.4 Analysis

Impact of correcting δ in the MILP baseline. The MILP baseline objective increases substantially with the δ correction:

Instance	Old (no δ)	New (strict δ)
R=10	4.79 (opt)	5.87 (opt)
R=20	200.78 (tLim)	334.75 (tLim)
R=30	aborted	aborted

For R=20, the strict MILP is dramatically harder: the incumbent found within 60 s is 334.75, worse than `topology_ms6` at 236.06. This is expected: the δ separation creates tight sequencing constraints with many aircraft per position, making the MIP tree much harder to prune.

Topology heuristic is now the strongest method for R=20/30. Under strict δ, `topology_ms6` (236.06 for R=20, 225.51 for R=30) is the best feasible method within 60 s for medium and large instances. The MILP baseline no longer provides a useful upper reference for R≥20.

Table 5: Strict-δ baselines (2026-05-14). All solutions pass RQ01–RQ08. Bold = best per instance.

Instance (R)	Method	Obj	Makespan	Delay	Mov	Time(s)
`scn_custom` (R=10)	milp_baseline	**5.87**	58.67	0.00	0	28.3
	topology_ms6	17.10	72.72	9.83	0	60.0
	tgr_k5	16.93	62.59	10.67	0	10.1
	fas_on_topo	**7.06***	65.25	0.54	0	0.0
`scn_few-loose` (R=3)	milp_baseline	**3.12**	31.22	0.00	0	0.3
	topology_ms6	**3.12**	31.22	0.00	0	60.0
	tgr_k5	**3.12**	31.22	0.00	0	10.0
	fas_on_topo	**3.12**	31.22	0.00	0	0.0
`scn_few-tight` (R=2)	milp_baseline	**6.39**	63.88	0.00	0	0.4
	topology_ms6	**6.39**	63.88	0.00	0	60.0
	tgr_k5	**6.39**	63.88	0.00	0	10.0
	fas_on_topo	**6.39**	63.88	0.00	0	0.0
`scn_heavy-tight` (R=30)	milp_baseline	aborted	—	—	—	—
	topology_ms6	**225.51**	247.74	200.73	0	81.4
	tgr_k5	254.90	251.11	229.79	0	60.1
	fas_on_topo	224.24†	247.74	199.46	0	60.2
`scn_many-medium` (R=20)	milp_baseline	334.75†	122.91	322.46	0	71.7
	topology_ms6	**236.06**	111.01	224.96	0	60.0
	tgr_k5	267.78	105.69	257.21	0	60.1
	fas_on_topo	**247.86**	111.01	236.76	0	13.3

FAS on topology assignment: R=20 regression resolved. In Series 03 (pre-fix), `fas_on_topo` gave 253.59 vs topology's 242.86 for R=20 — apparently worse. Under strict δ, topology gives 236.06 and `fas_on_topo` gives 247.86 — still worse, confirming that the regression persists. Root cause: FAS models blocking arcs at aircraft-block level ($D_r = \sum d_j$), which is more conservative than the topology heuristic's job-by-job blocking evaluation. This is a modelling gap in FAS, not a feasibility bug (see Section 6.5).

R=30: fas_on_topo nearly matches topology_ms6. Under strict δ, `fas_on_topo` achieves 224.24 vs topology's 225.51 for R=30 — effectively equal, with FAS not yet proven optimal within 60 s. FAS uses the same position assignment as the topology solution; any remaining gap is due to the blocking-arc conservatism noted above.

Small instances: all methods find the same optimum. For R≤5, all four methods achieve the same objective under strict δ, consistent with prior results.

11.5 Summary of Corrections Applied (2026-05-14)

1. **RQ08** added to `scripts/output_data/check_solution.py`. Unit tests in `scripts/tests/test_rq08.py` (6 cases, all pass).
2. **MILP Pyomo**: `cSamePositionDeltaOrder1/2` constraints added to `models/milp_pyomo.py`.
3. **Topology heuristic**: confirmed passes RQ08 with min gap = 10.00 on all instances — no code change needed.
4. **FAS model**: passes RQ08 by design (explicit ordering constraints with $+\delta$). Known conservatism in blocking-arc modelling documented in Section 6.5.

11.6 Next Steps

1. **Fix FAS blocking-arc conservatism**: use job-level timing for blocking arc constraints instead of aircraft-block duration D_r. Expected to close the topology vs fas_on_topo gap on R=20.
2. **Pipeline seguro topology→FAS**: return min(topology, fas_on_topo) to guarantee FAS never degrades the solution.
3. **tgr_long**: test TGR with 20 s per assignment ($K = 3$, `ls_mode="full"`) to recover performance on R=20/30.
4. $|C_r| = 2$ **selectivo**: after fixing FAS blocking arcs, add a second position candidate for high-delay aircraft to attack the residual gap R=10 (fas_on_topo=7.06 vs milp=5.87).